GEORGETOWN UNIVERSITY

Claude Code in Academic Research

Four Case Studies

Tiago Ventura

Georgetown University, McCourt School of Public Policy

2025

What is an Agentic Coding Model?

Drawing from Sol Messing's presentation — an agentic coding model is an LLM that:

- **Generates** code
- Can directly **edit files** in your codebase
- Can **execute** that code
- Can **evaluate** the results and iterate

Key distinction from a chatbot: it acts autonomously within your project — reading files, running commands, making changes — not just answering questions.

Examples: Claude Code, GitHub Copilot Agent, Cursor, Windsurf

A Framework for Evaluation

Three dimensions for assessing agentic coding models in research:

Dimension	Core Question
Research Productivity	Does it let me move faster, fail fast, scale?
Scientific Integrity	Hallucinations, AI slop, p-hacking temptation, false confidence?
Technical Capability	What can it handle? What's impressive vs. what falls short?

Key tension: the same speed that helps you fail fast enables p-hacking. Impressive output can mask errors.

I'll evaluate each case study through these three lenses.

Roadmap

Four case studies of how I use Claude Code in my research workflow:

1. Multi-Agent Tournament — Selecting a replication paper from 285 candidates
2. Data-to-Paper Pipeline — Full paper writing with Claude (that failed)
3. Coding Tasks, Plan Mode & Skills — Country pipelines, verification, and reusable workflows
4. Multi-Skill Peer Review — Four specialized review skills as an orchestrated system

For each, I'll assess: research productivity, scientific integrity, and technical capability.

Case Study 1

Multi-Agent Tournament

Selecting a replication paper from ~285 candidates

The Problem

I needed to find the best published survey experiment to replicate for the **Coppock & McGrath (2026)** replication competition.

- Search hundreds of papers across 8 political science journals
- Score them on a detailed rubric (statistical power, effect sizes, replicability, team fit)
- Make a **defensible, well-documented** decision
- Doing this manually would take weeks

Prompt Architecture: Phased Checkpoints

I wrote a ~900-line `instructions.md` with mandatory stop-and-check points:

```
Your task (in phases):
0. Build a deep understanding of the RFP call
1. Build a deep understanding of the research
   pipeline and co-authors
2. Search for high-impact survey experiments published
   in four leading political science journals (2010-2025)
3. Evaluate papers as candidates for a replication exercise
4. Run a multi-agent tournament between the top papers
```

Key rule:

```
## IMPORTANT: Stop-and-Check Points
At each of these points, you must:
1. Summarize what you have completed
2. Present key outputs for review
3. List any issues or concerns
4. Wait for human approval before proceeding

Do not proceed past a checkpoint without explicit approval.
```

View full prompt → replication_tournement/instructions.md

Prompt Refinement: Real Conversation Excerpts

The instruction file evolved through dialogue:

> *"Even though the RFP talks about replication + extension, I want your focus to be only on the replication"* — scoping
>
> *"Add as criteria to the soft scoring: a) experiment ran with low statistical power; b) effect sizes are larger than 0.3 sd"* — iterative rubric refinement
>
> *"Using multiple models (from OpenAI or Google) would improve this task?"* — multi-model idea
>
> *"Save for me two instructions.md files, one with the multi model competition, and one without it"* — multi-trial design

The Tournament Design

A **multi-agent debate format** with 3-judge panels:

Judge	Model	Role
Judge 1	Claude Opus	Native (Claude Code)
Judge 2	GPT-5	OpenAI API
Judge 3	Claude Sonnet	Anthropic API

20 Champion Agents (one per shortlisted paper) argue for their paper. Winners decided by **majority vote (2 of 3)**.

```
STAGE 1: INTRA-JOURNAL ELIMINATION (5 per journal)
  12 matches → 8 papers advance (2 per journal)

STAGE 2: CROSS-JOURNAL BRACKET
  Quarterfinals → Semifinals → Final
  7 matches → 1 champion

Total: 19 matches per trial
```

Output: Trial Results

I ran **3 independent trials** with different model/filter configurations:

Trial Design | Cross-Trial Consistency

	Trial 1	Trial 2	Trial 3
Scoring model	Claude Opus	Claude Opus	GPT-5
Judge model	Claude Opus	Claude Opus	GPT-5
Hard filters	H1-H5	H1-H7	H1-H7
Papers scored	285	263	263

Model Sensitivity

Each trial produced a **different champion paper**.

But P176 (Druckman et al. 2022) was the **only paper in the top 3 across all three trials**.

Qualitative pattern in judge disagreements:

> Claude favored **theoretical ambition**; GPT favored **execution certainty**

This is why multi-model judging matters — single-model results are not robust.

Case Study 1: Lessons

- Checkpoint architecture works: mandatory stops kept Claude on track across a very long task
- Multi-model judging reveals real disagreements: different models have different evaluation biases
- Sensitivity to model choice was higher than expected: each trial had a different champion
- Deep evaluations via web search sometimes hallucinated paper details: had to add explicit verification steps
- The prompt was ~900 lines — probably too long for most use cases, but necessary here

Case Study 2

Data-to-Paper Pipeline

Full paper writing with Claude — that failed (and that's the point)

The Problem

I had data on mobile internet coverage in Brazil (ANATEL regulatory data with a **30,000-inhabitant threshold** creating a natural experiment).

Goal: Estimate causal effects of internet speed on mental health using a **fuzzy regression discontinuity design**.

Produce a complete paper — from literature review to LaTeX.

Prompt Architecture: 7 Phases

```
Phase 1: Literature review
         (every citation must be confirmed real)
Phase 2: Build the instrument
         (first-stage RD plots, covariate balance, McCrary tests)
Phase 3: Download mental health outcome data from DataSUS
Phase 4: Merge datasets at municipality level
Phase 5: Estimate fuzzy RD using rdrobust in R
Phase 6: Write the paper in LaTeX
         (Significance Statement, Abstract, Intro,
          Results, Discussion, Methods)
Phase 7: Organize code and final validation
```

Key Design Choices

Two rules in the instruction file shaped the entire project:

Model selection rule:

> Use Opus for tasks requiring intellectual judgment — literature synthesis, evaluating replication strength, and all writing. Use Sonnet for coding tasks.

Null results handling:

> If the analysis finds no significant effects, write the paper with null findings.

This second rule turned out to be critical.

Prompt Refinement: Conversation Excerpts

> *"Please read @instructions.md file, and grill me on it"* — using Claude to stress-test my own research design before execution
>
> *"Which of your models do you suggest me to use for each of these tasks?"* — model-selection dialogue
>
> *"Instead of defining the model for each task, give a general recommendation to use Opus on tasks that require intellectual judgment and Sonnet on coding"* — the Opus/Sonnet split

The Estimation Mistake

DataSUS data downloaded successfully. Claude ran the first estimation — confidently, but **wrong**.

The instruction said: *"fuzzy RDD around the 30,000 population threshold."*

Claude used **30,000 as a bandwidth** (a smoothing parameter) instead of using it as the **cutoff** for the fuzzy RD.

This is a subtle but fundamental misunderstanding of the research design. Claude executed confidently and without flagging uncertainty.

Catching this required domain expertise — knowing what a fuzzy RD should look like.

After the Correction

I caught the mistake and corrected the approach.

Corrected estimation showed **null results**.

Claude delivered this much faster than doing it manually.

Project killed → moved to the `_Died` folder.

But Claude helped me fail fast.

`View full prompt` → _Died/AV_internet_speed_politics/instructions.md

P-Hacking on Steroids

The speed that helps you "fail fast" also creates a temptation:

- Claude makes it **trivially easy** to keep re-running specifications
- Tweak bandwidths, add covariates, try different outcomes — until something is significant
- The same tool that efficiently delivered null results could also be used to p-hack your way to a false positive
- The acceleration of the research pipeline amplifies both **good practices** (failing fast) and **bad ones** (specification searching)

Case Study 2: Lessons

- Domain expertise is essential: Claude executed the wrong estimation confidently — you must know what correct output looks like
- Failing fast has value: Claude helped me assess an idea's viability in days, not months
- P-hacking risk is real: the speed that helps you fail fast also makes specification searching trivially easy
- The Opus/Sonnet split is a useful heuristic: judgment vs. coding maps well to model strengths

Case Study 3

Coding Tasks, Plan Mode & Skills

Country pipelines, verification, and reusable workflows

CLAUDE.md: Instant Onboarding

The Global Social Media Study (23-country experiment, registered report at *Nature*) uses a `CLAUDE.md` that documents the project architecture:

- Language, dependencies, path management, country-folder pattern
- Numbered script pipeline (`00_` through `05_`)

Once in place, Claude could replicate the UK pilot pipeline across 23 countries — adapting for each while respecting shared utilities.

The `CLAUDE.md` replaced hours of onboarding with a single file read.

`View project docs` → gsm_second_stage_code/CLAUDE.md

Coding Tasks: Tests and Verification

Beyond pipeline replication, Claude handled tedious verification tasks:

- Running **consistency checks** on country data (e.g., verifying participant counts match across waves)
- Having Claude **verify if numbers** in code output match expectations
- **Testing if the pipeline works** correctly end-to-end before replicating across countries

These are tedious-but-critical tasks where Claude excels — freeing researcher time for design and interpretation.

Plan Mode

For complex analyses, Claude writes a **structured plan before coding**:

```
# Plan: Missing Data Robustness Analysis

### Module 1: Data Loading & Match Indicator
  Input: model_row_counts.csv, timelines parquet
  Steps: Load, clean, create matched indicator
  Test: Print dimensions, verify no unexpected NAs

### Module 2: Summary Statistics & Balance Table
### Module 3: Statistical Tests
### Module 4: Visualizations
### Module 5: Additional Analysis
```

Each module specifies inputs, steps, outputs, and verification.

Plan → Code

The plan produced `code/09_missing_data_analysis.R` — **541 lines, 5 modules**:

Module	Content	Output
1	Data Loading & Match Indicator	Analytical dataset
2	Balance Table	`balance_table.tex`
3	Statistical Tests	`logistic_regression_table.tex`
4	Visualizations	4 PDF figures
5	Additional Analysis	`fig_ideology_deciles.pdf`

Each module has **built-in verification steps** — Claude checks its own output before proceeding.

Plan Mode is valuable for non-trivial analyses where you want Claude to think through the structure before writing code.

View full plan → SVTR_twitter_ban/plans/missing_data_analysis.md

The APSR-Tables Skill

Academic papers need appendix regression tables, but main figures are visualizations. Converting figure code back into tables is tedious.

I built a reusable **Claude Code skill** (`/apsr-tables`):

```
Step 1: Scan the manuscript .tex for figures
Step 2: Locate figure-producing R code
        (ggsave, save_function, pdf, png)
Step 3: Trace back to the regression models
        (feols, lm, glm, fixest, ivreg)
Step 4: Classify each figure
        (model-based → needs table; descriptive → skip)
Step 5: Generate appendix tables using modelsummary
```

Invoke with `/apsr-tables` in any project

Building the Skill: Conversation History

> *"I would like your help to build a skill for me called apsr-tables"* — skill creation
>
> *"Please add this recommendation of using the .tex directly and avoid save_kable"* — iterative improvement based on real bugs

The key rule that emerged:

```
Save tables by passing the file path directly
to modelsummary's output argument:
  output = "path/to/table.tex"

Never use save_kable — it produces HTML in
interactive R sessions even when saving to .tex files.
```

Cross-Cutting Pattern

Three tools that work together:

1. CLAUDE.md — Project documentation that makes onboarding instant
2. Custom skills — Reusable workflows invocable with a slash command
3. Numbered pipelines — Sequential scripts that Claude can replicate across contexts

Case Study 3: Lessons

- CLAUDE.md makes onboarding instant: Claude reads it once and understands the project
- Plan Mode structures complex analyses: structured planning before coding produces well-organized scripts with built-in verification
- Skills save hours per paper: the `apsr-tables` skill can be invoked on any paper project
- Claude excels at repetitive-but-slightly-different tasks: country pipelines and verification checks are ideal
- Over-engineering risk: Claude sometimes builds abstractions when you need copy-paste-modify

Case Study 4

Multi-Skill Peer Review

Four specialized review skills as an orchestrated system

The Problem

Peer reviewing manuscripts requires checking multiple dimensions simultaneously:

- Substantive theory and causal inference
- Statistical methods and transparency
- Writing quality (typos, grammar, clarity)
- Consistency between code, figures, tables, and text

Asking Claude to "review this paper" produces muddled, unfocused feedback.

Architecture: Four Specialized Skills

```
/substantive-review ─┐
                     │
/technical-review ───┼──→ /meta-review (orchestrator)
                     │          ↓
/writing-review ─────┘     Unified prioritized
                           action plan
```

Each skill has a **narrow scope** and produces a separate markdown file.

The meta-reviewer runs all three, reads their outputs, and synthesizes.

Technical Review in Action

Concrete findings from `/technical-review` on a manuscript:

Clustering Issue | Numerical Error | Code Bug | Unverifiable Stats

> *"The paper mentions that errors are 'clustered at the user level'... the choice of clustering level affects the standard errors. The paper should justify why user-level clustering is sufficient and whether two-way clustering would be more appropriate."*

`View full review` → SVTR_twitter_ban/review/MAIN_MOST_UPDATED_technical_review.md

Separation of Concerns

The **writing reviewer** is explicitly scoped:

What to review | What NOT to review

```
- Typos and errors: grammar, spelling, punctuation,
  wrong words, subject-verb agreement, tense
  inconsistencies, missing articles, duplicated words
- Clarity suggestions: awkward sentences, unclear
  phrasing, unnecessarily complex constructions,
  poor flow between sentences
```

This prevents the "do everything at once" failure mode.

Meta-Reviewer Output

The meta-reviewer synthesizes a **prioritized action plan**:

```
## Submission Readiness Verdict
[Overall assessment: submit as-is, minor fixes,
 substantial revision, or major revision needed]

## Priority Action List

### Must Fix Before Submission
1. [Issue] [Substantive] — show-stoppers

### Should Fix
1. [Issue] [Technical] — high-impact improvements

### Optional / Nice to Have
1. [Issue] [Writing] — minor improvements

## Cross-Cutting Themes
- [Theme]: [Substantive] flagged X,
  while [Technical] flagged Y. Together,
  these suggest Z.
```

Reuse Across Projects

> *"/meta-review"* — first invocation on a manuscript
>
> *"I need you to edit the meta-review skill... actually call the three other reviews"* — upgrading to orchestrator pattern
>
> *"Now I need you to run the /meta-review in this project. However, adjust it given this grant call"* — reuse across different contexts (papers, grants)

The same skill system works for manuscripts, grant proposals, and working papers.

Case Study 4: Lessons

- Specialized > generalist: narrow-scope reviewers produce more thorough feedback
- The technical reviewer catches what humans miss: figure-text inconsistencies, table-code mismatches
- The orchestrator pattern works: run sub-skills, then synthesize
- Scope creep prevention matters: explicitly telling the writing reviewer NOT to comment on substance is essential
- The substantive reviewer sometimes hedges too much ("could be improved" instead of "this is wrong")

Cross-Cutting Lessons

Research Productivity

- Checkpoint architecture: mandatory stop-and-check points keep Claude on track across long tasks
- CLAUDE.md as documentation: a well-written project file makes Claude immediately productive
- Failing fast: Claude helps you assess viability quickly — not every project should survive
- Skills for repetitive workflows: the upfront cost of building a skill pays off across projects

Scientific Integrity

- Citation hallucination: Claude will invent plausible-sounding citations. Verification steps are mandatory.
- Domain expertise is non-negotiable: Claude executes wrong specifications confidently — you must know what correct output looks like
- Hedging in reviews: the substantive reviewer needs explicit instructions to be direct, not equivocate

Technical Capability

- Separation of concerns: narrow-scope skills outperform broad "do everything" prompts
- Multi-model validation: different models have different biases — use this to your advantage
- Plan Mode: structured planning before coding produces well-organized analysis scripts with built-in verification
- Over-engineering risk: Claude sometimes builds abstractions when you need simple copy-paste-modify

Practical Tips

1. **Start with `CLAUDE.md`** — invest 30 minutes documenting your project for Claude
2. **Use checkpoints** — don't let Claude run unsupervised across long tasks
3. **Build skills for repetitive workflows** — the upfront cost pays off across projects
4. **Separate judgment from coding** — Opus for thinking, Sonnet for executing
5. **Run multiple trials** — if the result is sensitive to model choice, you want to know
6. **Let projects die** — Claude helps you fail fast, and that's valuable

Thank you!

Tiago Ventura — Georgetown University